

Universidad Nacional de Colombia
Facultad de Ingeniería



Ejercicios

Presentado por:
Leal Diaz Layonel Camilo
Jaekle Till

Profesor:
José J. Martínez P.

Asignatura:
Inteligencia Artificial y mini-robots

Bogotá, D.C

24 de marzo de 2026

Ejercicio 1.

El código implementa una **Arquitectura de IA Agéntica** (Agentic AI). A diferencia de un chatbot tradicional que simplemente responde preguntas de forma lineal, este sistema orquesta un equipo de "agentes" virtuales. Cada agente tiene una personalidad definida, un rol específico y acceso a herramientas concretas. Los agentes interactúan entre sí, delegan tareas y toman decisiones autónomas para lograr un objetivo final: investigar un tema (usando documentos locales e internet) y generar un reporte en PDF.

2. Las Librerías Principales (El Motor)

Para construir este ecosistema, integramos varias librerías clave que trabajan en conjunto:

- **llama-index** (Específicamente el módulo de **Workflows**): Es la columna vertebral del proyecto. Actúa como el "tablero de control" que define las reglas de cómo los agentes se comunican entre sí y cómo se pasan el control (*handoffs*).
- **google-genai** y **llama-index-llms-google-genai**: Son los "cerebros" del sistema. Proveen la conexión a los modelos de lenguaje de Google (Gemini). En este caso, usamos la familia **gemini-2.5-flash** y **gemini-3-flash-preview** por su rapidez y capacidad para procesar archivos y ejecutar herramientas lógicas.
- **markdown-pdf**: Una librería utilitaria de Python. Toma el texto estructurado (Markdown) generado por la IA y lo compila en un documento PDF real y descargable.
- **asyncio** y **nest_asyncio**: Manejan la "asincronía". Como las llamadas a internet, la lectura de archivos y las respuestas de la IA toman tiempo, estas librerías permiten que el código espere las respuestas sin bloquear o congelar el entorno de Google Colab.

3. El Equipo de Trabajo (Los Agentes)

Diseñamos un flujo de trabajo con tres roles distintos para optimizar el proceso y evitar que un solo modelo se confunda intentando hacer todo a la vez:

1. **SupervisorAgent (El Orquestador):**
 - **Rol:** Es el gerente del proyecto. Recibe la instrucción inicial del usuario, pero tiene una regla estricta: *no puede hacer el trabajo por sí mismo*.
 - **Función:** Analiza la petición y usa una herramienta de delegación interna para pasarle las instrucciones exactas y el nombre del archivo al investigador.
2. **ResearchAgent (El Investigador):**
 - **Rol:** El operario de recolección de datos.
 - **Función:** Tiene acceso a las herramientas de lectura y búsqueda. Lee el PDF que el usuario subió y busca en Google cualquier contexto que falte. Una vez que junta toda la información cruda, se la pasa al redactor.
3. **WriterAgent (El Redactor):**
 - **Rol:** El especialista en formato.
 - **Función:** Toma los datos desordenados del investigador, los redacta como un reporte profesional en formato Markdown y ejecuta la herramienta de creación de PDF. Al terminar, le avisa al sistema que el trabajo concluyó.

4. Las Herramientas (Tools)

Los agentes por sí solos son solo generadores de texto. Para interactuar con el mundo real, les construimos "brazos robóticos" o herramientas:

- **read_local_pdf**: Una función que toma un archivo PDF local, lo convierte en bytes y

se lo inyecta directamente a la API de Gemini (Inyección de Contexto). Es ideal para extraer datos precisos de documentos.

- **search_web**: Utiliza la capacidad nativa de Gemini para conectarse a Google Search. Permite que el sistema valide datos o busque información actualizada en tiempo real que no esté en el PDF.
- **generate_final_pdf**: Una función puente que recibe el texto final de la IA y usa la librería **markdown-pdf** para materializarlo en un archivo físico en el disco duro.

5. El Bucle de Ejecución y "Rayos X"

La parte final del código (**async def main()**) es el motor que pone todo en marcha.

Implementamos un sistema de monitoreo o "Rayos X" (**handler.stream_events()**). En lugar de ocultar el proceso, el sistema intercepta e imprime en la consola cada evento interno que ocurre en LlamaIndex. Esto permite observar en tiempo real cuándo un agente toma el control, qué está pensando (**Razonamiento Interno**), qué herramienta decide ejecutar y cuáles son los resultados de esa herramienta, haciendo que la arquitectura sea completamente transparente y excelente para fines educativos.

Ejercicio 2.

Visión General del Sistema

Este código implementa un sistema **RAG (Retrieval-Augmented Generation)**. A diferencia de preguntarle a ChatGPT o Gemini sobre un tema general y esperar que lo recuerde de su entrenamiento original, un sistema RAG le da a la IA un "libro abierto" con información específica y actualizada (en este caso, una página web) para que lea *antes* de responder.

Le llamamos **Híbrido** porque divide el trabajo en dos partes para optimizar costos y evitar caídas del servidor: usa el procesamiento de tu propia computadora (local) para buscar la información, y usa la nube de Google (Gemini) únicamente para redactar la respuesta final.

2. Las Librerías Principales (El Motor)

Para construir este ecosistema, conectamos diferentes tecnologías que se especializan en tareas únicas:

- **langchain** (y su ecosistema **core, community**): Es el framework principal. Si en el ejercicio 1 LlamaIndex era el director de una fábrica de agentes, aquí LangChain es la "tubería" que conecta el flujo de datos desde la página web hasta la respuesta final.
- **beautifulsoup4**: Es el extractor. Se encarga de entrar a la URL proporcionada, limpiar el código HTML y extraer únicamente el texto legible.
- **sentence-transformers** y **langchain-huggingface**: Son el traductor matemático. Toman el texto humano y lo convierten en números (Vectores/Embeddings) usando un modelo gratuito y local (**all-MiniLM-L6-v2**) que se ejecuta en la memoria de la máquina sin consumir cuota de internet.
- **faiss-cpu**: Es la base de datos de Facebook AI. Actúa como un archivero ultrarrápido que guarda los vectores matemáticos y busca similitudes casi instantáneamente.
- **langchain_google_genai**: Es la conexión con **Gemini 2.5 Flash**, que actúa puramente como el "Cerebro Lector" que analiza los fragmentos recuperados y redacta una respuesta coherente.

3. El Flujo de Trabajo (Paso a Paso)

El sistema opera en una secuencia lógica muy similar al procesamiento de información en ingeniería:

1. **Extracción (Web Scraping):** El sistema lee todo el texto de la página web indicada.
2. **Fragmentación (Chunking):** La IA no puede leer un libro entero de golpe sin saturarse. Por eso, dividimos el texto gigante de la página web en "fragmentos" (**chunks**) de 1000 caracteres, con un pequeño solapamiento de 200 caracteres para no cortar ideas por la mitad.
3. **Vectorización e Indexación:** Traducimos cada fragmento a coordenadas matemáticas. Por ejemplo, si el sistema procesa la malla curricular de Ingeniería Industrial, agrupará matemáticamente los párrafos que hablen de "optimización" cerca de los que hablen de "procesos", permitiendo búsquedas por concepto y no solo por palabra exacta. Todo esto se guarda en FAISS.
4. **Recuperación (Retrieval):** Cuando el usuario hace una pregunta, el sistema busca en FAISS los 3 fragmentos matemáticamente más parecidos a la pregunta.
5. **Generación Aumentada:** El sistema inyecta esos 3 fragmentos en el *prompt* oculto y le dice a Gemini: *"Responde a la pregunta usando SOLO este texto"*.

4. La Magia de LCEL (La Arquitectura Moderna)

La innovación técnica de este código radica en el uso de **LCEL (LangChain Expression Language)**. En lugar de usar funciones complejas y anidadas, el "cerebro" del sistema se construye con tuberías (el símbolo `|`):

Python

None

```
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```

Esto se lee literalmente como una línea de ensamblaje: *Toma la pregunta del usuario -> Pásala por el recuperador para buscar el contexto -> Une ambos en el formato del Prompt -> Envíalo al modelo Gemini (LLM) -> Extrae la respuesta en texto limpio.*

5. Transparencia y Antialucinaciones

Para la clase, este sistema tiene dos características vitales:

- **Temperatura 0:** Obliga a Gemini a ser estrictamente determinista y analítico. Si la respuesta no está en el texto de la universidad, el modelo admitirá que no lo sabe, evitando la "alucinación" (inventar datos).
- **Modo Rayos X:** Al igual que en el ejercicio anterior, el bucle **while** está diseñado para imprimir en pantalla los **[FRAGMENTOS USADOS]** antes de dar la respuesta. Esto permite auditar de dónde sacó la IA la información, demostrando que está leyendo la web y no adivinando.

Ejercicio 4.

Explorando la Frontera de la IA Física en Colombia: El Caso de Gemini Robotics-ER 1.5

Introducción: De la Pantalla al Mundo Real

Durante mucho tiempo, hemos interactuado con la Inteligencia Artificial a través de pantallas: escribiendo textos, analizando datos o generando imágenes. Sin embargo, el siguiente gran salto tecnológico no está en el software aislado, sino en la **Inteligencia Artificial Física (Embodied AI)**. Esto significa dotar a las máquinas y robots de la capacidad de entender, razonar y actuar en nuestro mundo físico.

En Colombia, un país con retos significativos en manufactura, logística, agricultura y atención médica, la implementación de IA Física no es solo una curiosidad científica, sino una necesidad estratégica para mejorar nuestra competitividad. Para entender este potencial, debemos analizar las herramientas que están haciendo esto posible hoy, como el nuevo modelo **Gemini Robotics-ER 1.5**.

¿Qué es Gemini Robotics-ER 1.5 y por qué es revolucionario?

Gemini Robotics-ER 1.5 no es un simple programa de control de motores. Es un **Modelo de Visión-Lenguaje (VLM)** diseñado específicamente para llevar capacidades agénticas a la robótica. En términos sencillos, permite que un robot "vea" su entorno a través de cámaras, "escuche" instrucciones en lenguaje natural y, lo más importante, "razone" sobre cómo interactuar con los objetos frente a él.

Las características que hacen de este modelo un punto de inflexión son:

- **Autonomía Mejorada:** Los robots ya no necesitan estar programados milimétricamente para una sola tarea repetitiva. Pueden adaptarse a entornos abiertos y cambiantes.
- **Interacción Natural:** En lugar de escribir código complejo, un operario humano puede simplemente decirle al robot: *"Encuentra la caja roja y ponla sobre la mesa"*.
- **Orquestación de Tareas:** El modelo tiene la capacidad de tomar una orden compleja (ej. "Prepara la estación de trabajo") y descomponerla en subtareas lógicas (buscar herramientas, despejar el área, mover materiales).

Posibilidades de Aplicación en Colombia

Como estudiantes de ingeniería, es nuestro deber proyectar cómo esta tecnología puede resolver problemas en nuestro contexto nacional:

1. Logística y Almacenamiento Dinámico

Los almacenes en Colombia a menudo enfrentan problemas de organización no estructurada. Usando la capacidad de Gemini Robotics-ER 1.5 para la **detección de objetos y cajas delimitadoras (bounding boxes)**, un robot logístico puede identificar inventario mixto en tiempo real, rastrear la ubicación exacta de productos mediante coordenadas 2D y moverlos de forma autónoma, incluso si el entorno cambió desde el día anterior.

2. Manufactura Flexible y Ensamblaje

La industria manufacturera colombiana se beneficiaría enormemente de la capacidad de

Planificación de Trayectorias de este modelo. En una línea de ensamblaje, el robot no solo detecta una pieza defectuosa, sino que el modelo de IA calcula instantáneamente la serie de movimientos (coordenadas) necesarios para que el brazo robótico extraiga la pieza sin chocar con el resto de la maquinaria.

3. Asistencia Robótica y Razonamiento Espacial

Pensemos en entornos más complejos como la agroindustria o la asistencia médica. El modelo puede realizar **razonamiento espacial y temporal**. Si se le pide a un robot agrícola que recolecte frutos maduros, Gemini evalúa la escena, identifica qué hojas o ramas debe apartar primero para hacer espacio, y ejecuta el movimiento mediante la llamada a APIs de control personalizadas (como abrir o cerrar pinzas).

Retos y Consideraciones Éticas en nuestro País

La adopción de la IA Física no está exenta de desafíos que debemos abordar como futuros ingenieros:

- **Seguridad Física:** Como advierte Google DeepMind, los modelos generativos pueden cometer errores (alucinaciones), y en el mundo físico, un error significa un robot chocando y causando daños reales. La seguridad industrial debe rediseñarse alrededor de estos sistemas autónomos.
- **Privacidad de los Datos:** Estos robots dependen de cámaras y micrófonos constantemente activos. Su implementación en fábricas u hospitales colombianos requerirá marcos legales estrictos para proteger la privacidad biométrica de los trabajadores y pacientes, asegurando técnicas como el desenfoque facial automático.
- **Latencia y Conectividad:** El modelo ofrece un "presupuesto de pensamiento" (thinking budget) flexible, permitiendo equilibrar la velocidad de reacción con la precisión. Sin embargo, esto requiere infraestructura de red robusta, un reto importante en zonas industriales alejadas de las ciudades principales.

Conclusión

El estudio de la IA Física mediante modelos como Gemini Robotics-ER 1.5 nos muestra que el futuro de la automatización ya no depende de la programación rígida, sino del razonamiento autónomo. Para Colombia, investigar y adoptar estas tecnologías no es ciencia ficción; es el camino hacia una industria más resiliente, adaptable y eficiente. Es nuestro turno, desde la academia, empezar a diseñar los algoritmos que conectarán la inteligencia artificial con nuestra realidad física.